MICROSAR I2C

Technical Reference

General Manual

Version 2.01.02

Authors	Arne van Mourik
Status	Released

Document Information

History

Author	Date	Version	Remarks
Arne van Mourik	2013-05-24	1.00.00	Initial version of this technical reference.
Arne van Mourik	2014-11-07	2.00.00	New version which describes the sequence concept of the driver.
Arne van Mourik	2014-12-17	2.00.01	Fixed architecture picture. Fixed include structure. Compressed chapter 6.
Arne van Mourik	2015-03-04	2.01.00	Added I2c stability features for communication protection against external disturbances.
Arne van Mourik	2015-03-31	2.01.01	Split the documentation into a general and a supplemental part. Added additional description of the channel purpose.
Arne van Mourik	2015-12-07	2.01.02	Document clean up. Added additional information about transfers.

Reference Documents

No.	Source	Title	Version
[1]		AUTOSAR_SWS_DevelopmentErrorTracer.pdf	3.2.0
[2]		AUTOSAR_SWS_DiagnosticEventManager.pdf	4.2.0
[3]		AUTOSAR_TR_BSWModuleList.pdf	1.6.0
[4]		AUTOSAR AUTOSAR_SWS_ECUStateManager.pdf	3.0.0
[5]		AUTOSAR AUTOSAR_SWS_BSWModeManager.pdf	1.2.0



Caution
We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector´s release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

Contents

1 Component History 6

2 Introduction 7

2.1 Architecture Overview 8

3 Functional Description 12

3.1 Features 12

3.1.1 Additions/ Extensions 12

3.1.1.1 Parameter Checking 13

3.2 Initialization 13

3.3 States 14

3.4 Main Functions 14

3.5 Error Handling 14

3.5.1 Development Error Reporting 14

3.5.2 Production Code Error Reporting 15

4 Integration 17

4.1 Scope of Delivery 17

4.1.1 Static Files 17

4.1.2 Dynamic Files 17

4.2 Compiler Abstraction and Memory Mapping 18

4.3 Critical Sections 19

4.4 Dependencies on SW modules 19

4.4.1 DET 19

4.4.2 DEM 19

4.4.3 MCU 19

4.4.4 PORT 19

4.4.5 SchM (Optional) 19

4.4.6 Dependencies on HW modules 19

5 API Description 20

5.1 Type Definitions 20

5.2 Interrupt Service Routines provided by I2C 22

5.3 Services provided by I2C 22

5.3.1 I2c_Init 22

5.3.2 I2c_DeInit 22

5.3.3 I2c_SetupEB 23

5.3.4 I2c_AsyncTransmit 24

5.3.5 I2c_GetStatus 24

5.3.6 I2c_GetResult ... 25
5.3.7 I2c_GetSequenceResult ... 25
5.3.8 I2c_Cancel ... 26
5.3.9 I2c_GetVersionInfo ... 26
5.3.10 I2c_SetHandling ... 27
5.3.11 I2c_MainFunction ... 27
5.4 Services used by I2C ... 28
5.5 Callback Functions ... 28
5.6 Configurable Interfaces ... 28
5.6.1 Notifications ... 28
5.6.1.1 I2c_SequenceEndNotification ... 28
5.6.1.2 I2c_SequenceErrorNotification ... 29

6 Configuration ... 30
6.1 Configuration Variants ... 30
6.2 Configuration with DaVinci Configurator Pro ... 30
6.2.1 Start configuration of I2C ... 30
6.2.1.1 Configuration sets ... 31
6.2.1.2 Channels ... 32
6.2.1.3 Channel lists ... 32
6.2.1.4 Sequences ... 33
6.2.2 Communication example ... 34
6.2.2.1 Logged transaction according to the given example ... 35
6.2.2.2 Possibilities of the driver (transfer examples). ... 35

7 Glossary and Abbreviations ... 42
7.1 Deviations ... 42
7.2 Additions/ Extensions ... 42
7.2.1 I2C sequence queue feature ... 42
7.2.2 I2C chain channel feature ... 42
7.2.3 I2C extended addressing ... 42
7.3 Glossary ... 42
7.4 Abbreviations ... 42

8 Contact ... 44

Illustrations

Figure 2-1 AUTOSAR 4.2 Architecture Overview 8
Figure 2-2 AUTOSAR 4.1 Architecture Overview 9
Figure 2-3 AUTOSAR 4.x Architecture Overview 10
Figure 2-4 AUTOSAR 3.x Architecture Overview 10
Figure 2-5 Interfaces to adjacent modules of the I2C 11
Figure 6-1 Configurator Basic Editor View. 30
Figure 6-2 Parameter description 31
Figure 6-3 Configure multiple configuration sets 31
Figure 6-4 Channel lists 32
Figure 6-5 PAGE WRITE transaction. 35
Figure 6-6 Data polling 35
Figure 6-7 RANDOM READ transaction 35

Tables

Table 1-1 Component history 6
Table 3-1 Supported AUTOSAR standard conform features 12
Table 3-2 Additional features provided by Vector 12
Table 3-3 Development Error Reporting: Assignment of checks to services 13
Table 3-4 Service IDs 15
Table 3-5 Errors reported to DET 15
Table 3-6 Errors reported to DEM 16
Table 4-1 Static files 17
Table 4-2 Generated files 17
Table 4-3 Compiler abstraction and memory mapping 18
Table 5-1 Type definitions 21
Table 5-2 I2c_ConfigType 21
Table 5-3 I2c_Init 22
Table 5-4 I2c_DeInit 23
Table 5-5 I2c_SetupEB 24
Table 5-6 I2c_AsyncTransmit 24
Table 5-7 I2c_GetStatus 25
Table 5-8 I2c_GetResult 25
Table 5-9 I2c_Cancel 26
Table 5-10 I2c_GetVersionInfo 27
Table 5-11 I2c_MainFunction 28
Table 5-12 Services used by the I2C 28
Table 5-13 I2c_SequenceEndNotification 29
Table 5-14 I2c_SequenceEndNotification 29
Table 7-1 Glossary 42
Table 7-2 Abbreviations 43

1 Component History

The component history gives an overview over the important milestones that are supported in the different versions of the component.

Component Version	New Features
1.00.00	Initial creation of a simple AUTOSAR like I2C driver.
2.00.00	Add the sequence concept for better channel handling.
2.01.00	Added I2c stability features for communication protection against external disturbances.

Table 1-1 Component history

2 Introduction

This document describes the functionality, API and configuration of the BSW module I2C. AUTOSAR does not specify a BSW component for I2C bus. The I2C component is a proprietary Vector extension of AUTOSAR BSW. The I2C driver was designed similar to the SPI driver and it is compatible to AUTOSAR 3.x and 4.0.3.

Supported AUTOSAR Release*:	4.0.3	
Supported Configuration Variants:	pre-compile	
Vendor ID:	I2C_VENDOR_ID	30 decimal (= Vector-Informatik, according to HIS)
Module ID:	I2C_MODULE_ID	84 decimal

* For the detailed functional specification please also refer to the corresponding AUTOSAR SWS.

This document describes the hardware independent functionality, configuration and the API of the I2C driver. The I2C driver provides services for basic communication with external components. These components can be used by an application.

The main tasks of the I2C are:

> Handle the I2C hardware units onboard.

> Handle data transmission to the components connected via I2C.

Take care of the settings required by external components (bit rate etc.)

2.1 Architecture Overview

The following figure shows where the I2C is located in the AUTOSAR architecture.

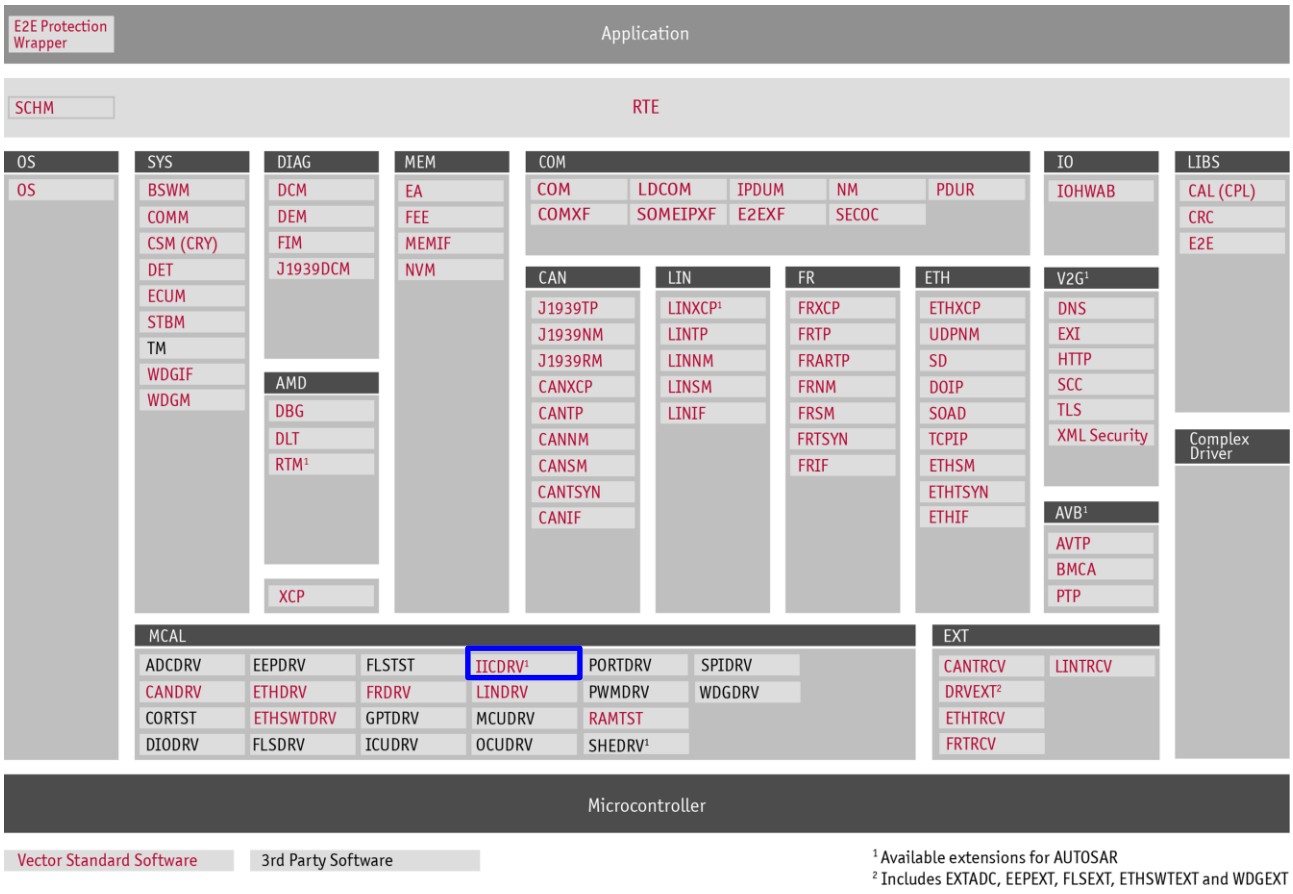


Figure 2-1 AUTOSAR 4.2 Architecture Overview

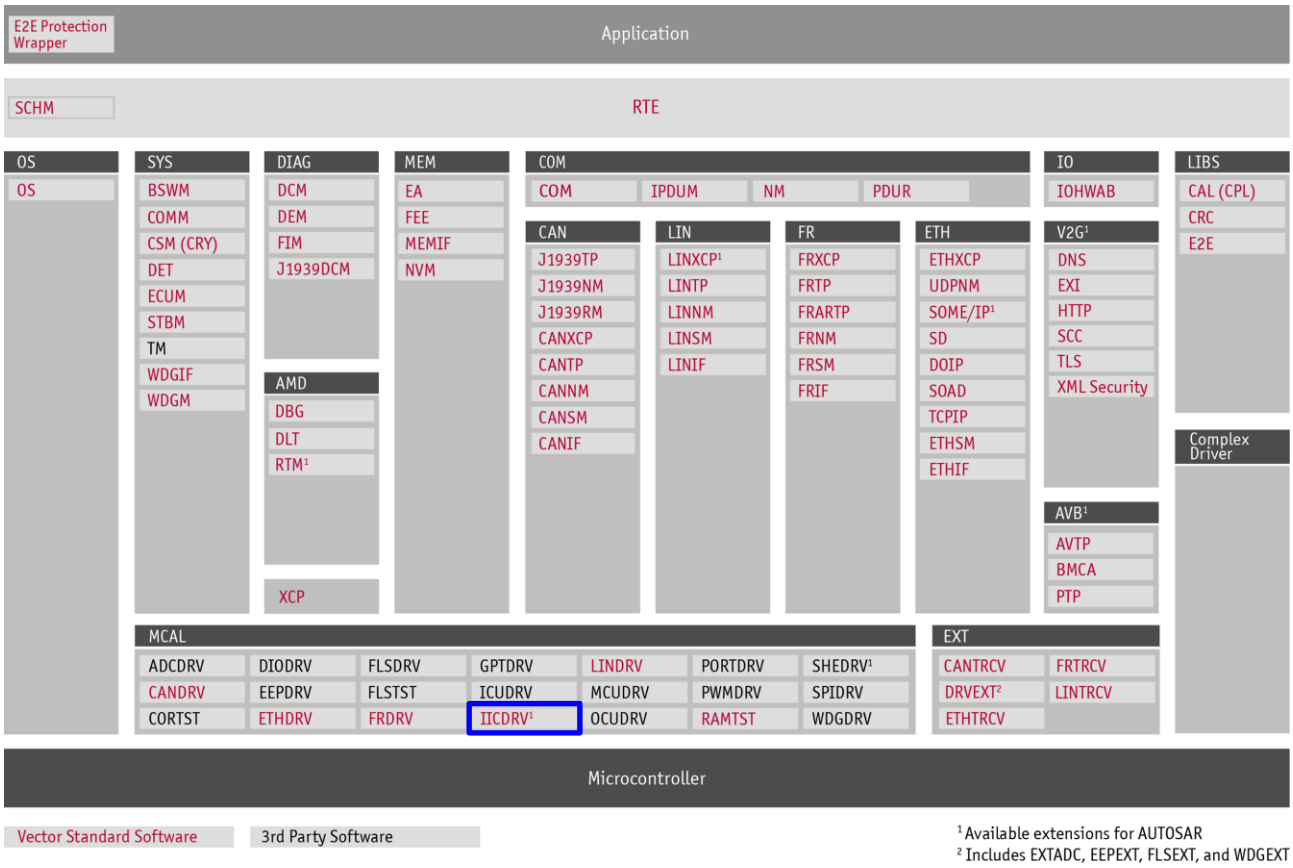


Figure 2-2 AUTOSAR 4.1 Architecture Overview

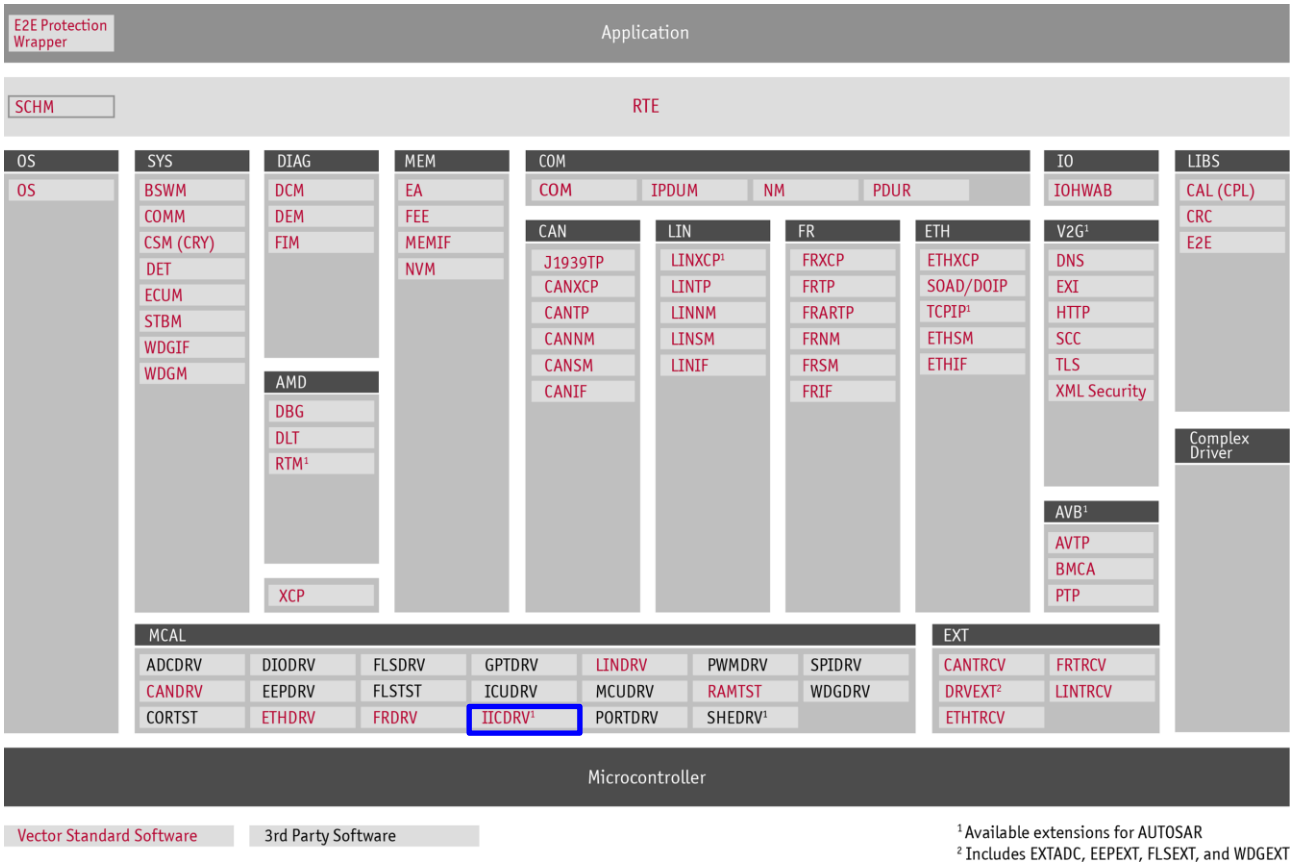


Figure 2-3 AUTOSAR 4.x Architecture Overview

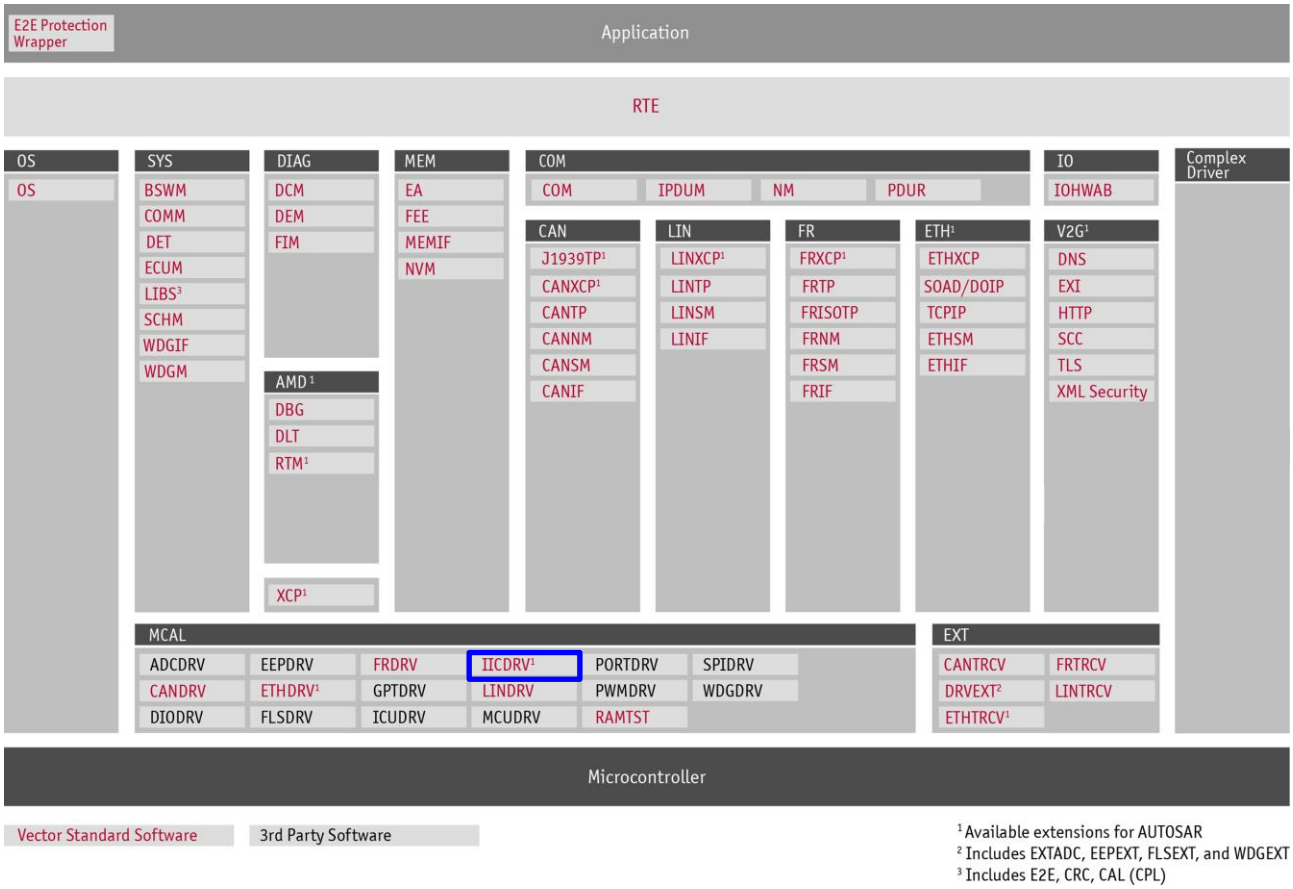


Figure 2-4 AUTOSAR 3.x Architecture Overview

The next figure shows the interfaces to adjacent modules of the I2C. These interfaces are described in chapter 5.

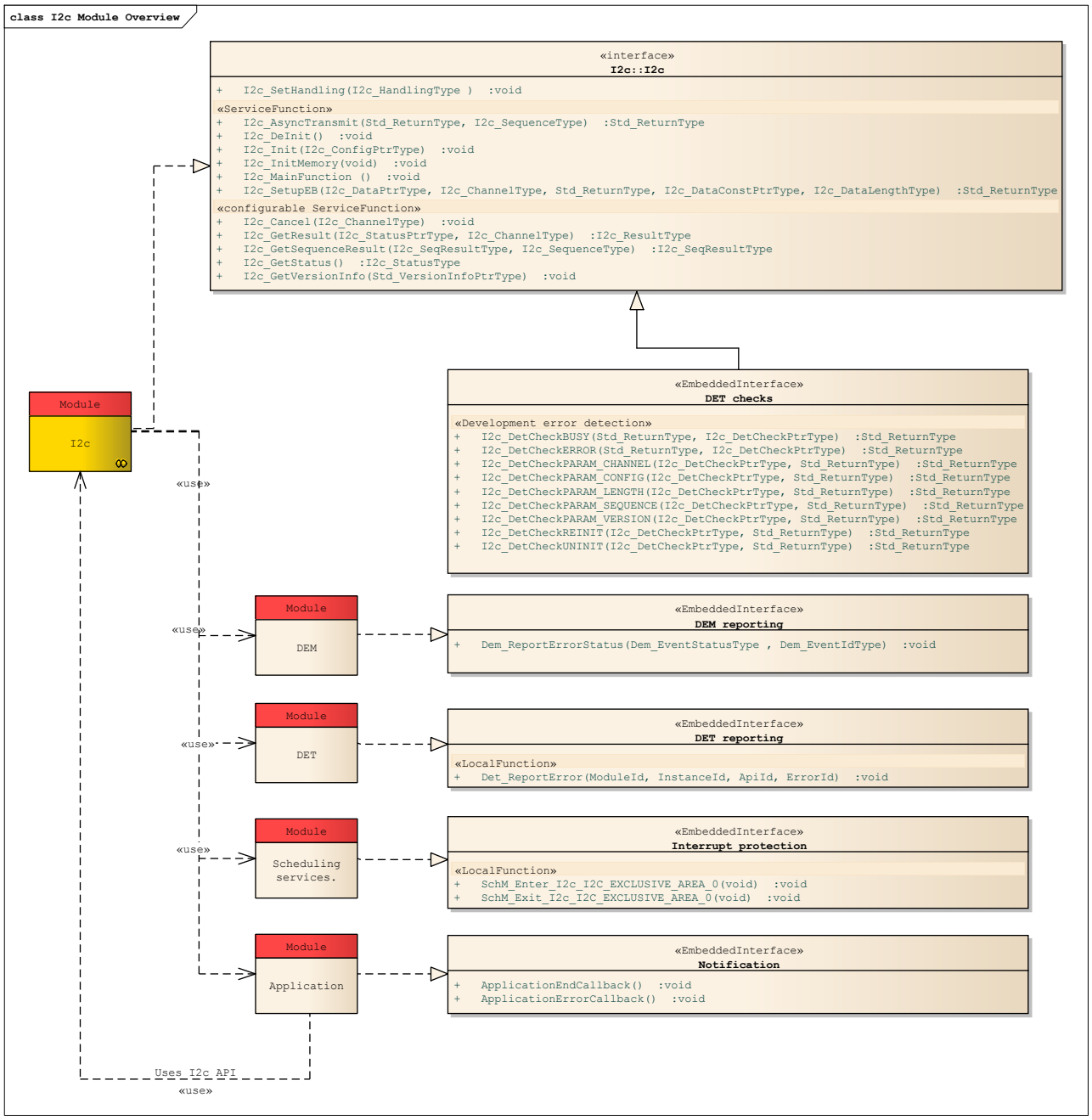


Figure 2-5 Interfaces to adjacent modules of the I2C

Applications do not access the services of the BSW modules directly. They use the service ports provided by the BSW modules via the RTE. The service ports provided by the I2C are listed in chapter 5.3.

3 Functional Description

3.1 Features

The features listed in the following tables cover the complete functionality specified by Vector for the I2C.

The following features are analog to AUTOSAR as specified by Vector:

Supported AUTOSAR Standard Conform Features
Configure I2C with: > Channel container. > Sequence container. > General container.
> Configure error detection (DET).
> Configure production error detection (DEM).
> Configure I2C interrupts and critical section protection.
> Configure API.

Table 3-1 Supported AUTOSAR standard conform features

3.1.1 Additions/ Extensions

The following extension provided by the driver:

Features Provided Beyond The AUTOSAR Standard
Hardware unit container to handle different hardware units.
Bus container to handle transfer speeds and hardware unit mapping.
Parallel sequence and channel processing on different hardware units.
Configure acknowledge ant not acknowledge handling (ACK, NACK).
Configurable addressing for standard (7bit) and extended (10bit) addresses.
I2C chain channel feature.
I2C sequence queuing feature.
I2C support for parallel handling of hardware units if more than one is available.
I2C improved error handling like: ▶ Handling for SDA stuck scenario. ▶ Handling for SCL low held scenario. ▶ Handling for general communication timeout.

Table 3-2 Additional features provided by Vector

3.1.1.1 Parameter Checking

The internal parameter checks of the API functions can be en-/disabled separately. The AUTOSAR standard requires en-/disabling of the complete parameter checking only. The next table shows the assignment of checks to the corresponding functions.

Service \ Check	I2C_E_UNINIT	I2C_E_ALREADY_INITIALIZED	I2C_E_BUSY	I2C_E_PARAM_CHANNEL	I2C_E_PARAM_SEQUENCE	I2C_E_PARAM_LENGTH	I2C_E_PARAM_VINFO_PTR	I2C_E_PARAM_CONFIG
I2c_Init		■	■					■
I2c_DeInit	■		■					
I2c_SetupEB	■			■		■		
I2c_AsyncTransmit	■				■			
I2c_GetStatus								
I2c_GetResult	■			■				
I2c_GetSequenceResult	■				■			
I2c_GetVersionInfo							■	
I2c_Cancel	■				■			
I2c_MainFunction	■							
I2c_SetHandling								

Table 3-3 Development Error Reporting: Assignment of checks to services

3.2 Initialization

The driver I2C is initialized by calling `I2c_Init()` with a pointer to a configuration as parameter. To re-initialize call `I2c_DeInit()` and then `I2c_Init()` again with a different configuration. The service for de-initialization is provided with the function `I2c_DeInit()`.



Caution
Take care to evaluate the return value of `I2c_DeInit`. The service is declined if the driver is active.

3.3 States

The I2C maintains states for:

- the driver,
- the configured channels,
- the sequences,
- and the hardware units.

The states can be obtained by the following API's:

- `I2c_GetStatus()` to get the driver status.
- `I2c_GetResult()` to obtain the channel result.
- `I2c_GetSequenceResult()` to obtain the sequence result.
- Hardware states can be obtained by extracting the upper byte of the Status parameter of the `I2c_GetResult` service.

3.4 Main Functions

The `I2c_MainFunction()` is provided and it was designed for an execution in a cyclic time raster (i.e. 10ms time raster of the OS).

3.5 Error Handling

3.5.1 Development Error Reporting

By default, development errors are reported to the DET using the service `Det_ReportError()` as specified in [1]. If development error reporting is enabled (i.e. pre-compile parameter `I2C_DEV_ERROR_DETECT==STD_ON`).

If another module is used for development error reporting, the function prototype for reporting the error can be configured by the integrator, but must have the same signature as the service `Det_ReportError()`.

The reported I2C ID is 84.

The reported service IDs identify the services which are described in 5.3. The following table presents the service IDs and the related services:

Service ID	Service	
I2C_SID_INIT	= 0x00	I2c_Init
I2C_SID_DEINIT	= 0x01	I2c_DeInit
I2C_SID_SETUPEB	= 0x02	I2c_SetupEB
I2C_SID_ASYNCTRANSMIT	= 0x03	I2c_AsyncTransmit
I2C_SID_GET_SEQUENCERESULT	= 0x04	I2c_GetSequenceResult
I2C_SID_GET_RESULT	= 0x05	I2c_GetResult
I2C_SID_CANCEL	= 0x06	I2c_Cancel
I2C_SID_GETVERSION_INFO	= 0x07	I2c_GetVersionInfo
I2C_SID_MAINFUNCTION	= 0x08	I2c_MainFunction
I2C_SID_SET_HANDLING	= 0x09	I2c_SetHandling

Table 3-4 Service IDs

The errors reported to DET are described in the following table:

Error Code	Description
[0x01] I2C_E_UNINIT	Error code for not initialized module.
[0x02] I2C_E_ALREADY_INITIALIZED	`Init` service called twice without `DeInit`.
[0x03] I2C_E_BUSY	Driver is busy.
[0x04] I2C_E_PARAM_CHANNEL	Channel out of bounds, exceeds the maximum number of configured channels.
[0x05] I2C_E_PARAM_SEQUENCE	Sequence out of bounds, exceeds the maximum number of configured sequences.
[0x07] I2C_E_PARAM_LENGTH	Length out of bounds – a zero length is not accepted. The total length is limited to 65535.
[0x08] I2C_E_PARAM_VERSIONPTR	An invalid version info pointer has been passed (a NULL_PTR).
[0x09] I2C_E_PARAM_CONFIG	An invalid configuration has been passed (a NULL_PTR).

Table 3-5 Errors reported to DET

3.5.2 Production Code Error Reporting

By default, production code related errors are reported to the DEM using the service `Dem_ReportErrorStatus()` as specified in [2]. If production error reporting is enabled (i.e. pre-compile parameter `I2C_PROD_ERROR_DETECT==STD_ON`).

If another module is used for production code error reporting, the function prototype for reporting the error can be configured by the integrator, but must have the same signature as the service `Dem_ReportErrorStatus()`.

The errors reported to DEM are described in the following table:

Error Code	Description
I2C_E_ADDRESSING_FAILURE	Error code is reported if the I2C detects an invalid address. This happens if a slave is addressed and the NACK response is configured as error.
I2C_E_ARBITRATION_FAILURE	Error is reported if the master loses arbitration. This usually happens if the SDA is stuck low.
I2C_E_BUS_FAILURE	Error is reported if the SCL line is stuck low. Only a reset can resolve this error. This error is also reported if the Main function service detects a communication timeout.

Table 3-6 Errors reported to DEM



FAQ

The error I2C_E_ARBITRATION_FAILURE is reported if the “Arbitration loss” is enabled. Otherwise the driver reports an I2C_E_BUS_FAILURE and tries the “bus clock free” strategy. If using “clock free”, the driver attempts to send nine clock pulses to the slave in order to synchronize the bus again. Due to the I2C specification a compatible device must release the SDA line within these nine clock pulses.

The I2C_E_BUS_FAILURE may occur after every power on of the hardware if a slave is not ready yet for communication. In this case the driver sends, after a timeout, the nine clock cycles within these the slave shall be synchronized with the bus again. This procedure continues as long the SDA is low.

The driver is capable to recover from these errors if the SDA or SCL lines return to their specified states (high level).

A bus error with SDA stuck low is reported after the driver was able to recover from this state (SDA was released) and is ready for communication again. If the line is not released due to a device or bus malfunction, the driver repeats the clock pulses every time a timeout is detected!

A bus error with SCL stuck low is reported at once if the driver detects a SCL timeout.

4 Integration

This chapter gives necessary information for the integration of the MICROSAR I2C into an application environment of an ECU.

4.1 Scope of Delivery

The delivery of the I2C contains the files which are described in the chapters 4.1.1 and 4.1.2:

4.1.1 Static Files

File Name	Description
I2c.c	Source file that contains the API relevant implementation.
I2c.h	Header file that contains all API relevant declarations.
I2c_Checks.c	DET checking routines for API.
I2c_Checks.h	Definition of the DET checks routines.
I2c_Dlc.c	Driver layer, handles the sequence, the channels and all hardware related tasks.
I2c_Dlc.h	Contains the type and function declarations for the I2c_Dlc module.
I2c_Irq.c	Source file that contains the ISR implementation.
I2c_Irq.h	Header file that contains the declarations of the ISR handlers.
DrvI2c__coreAsr.jar	Generator plugin.
I2c_bswmd.arxml	Basic Software Module Description according to AUTOSAR.
I2c_bswmd_Pre	Pre-configuration of the module
I2c_bswmd_Rec	Recommended configuration of the module.

Table 4-1 Static files

4.1.2 Dynamic Files

The dynamic files are generated by the configuration tool.

File Name	Description
I2c_Cfg.h	This file contains the definitions and mappings for the driver like: > Error detection setting. > Mapping of the functions of DET. The mapping is done by a macro which is empty if DET is off and otherwise points to the Det_ReportError if DET is on. > Channel, bus and Hardware unit definitions.
I2c_PBcfg.c	This file contains the assignments of channels, buses and hardware units. It provides runtime(s) for the driver to operate.

Table 4-2 Generated files

4.2 Compiler Abstraction and Memory Mapping

The objects (e.g. variables, functions, constants) are declared by compiler independent definitions – the compiler abstraction definitions. Each compiler abstraction definition is assigned to a memory section.

The following table contains the memory section names and the compiler abstraction definitions of the I2C and illustrates their assignment among each other.

Compiler Abstraction Definitions / Memory Mapping Sections	I2C_CODE	I2C_VAR1	I2C_VAR_NOINIT	I2C_VAR_INIT	I2C_VAR_ZERO_INIT	I2C_CONST	I2C_PBCFG	I2C_APPL_CODE	I2C_APPL_DATA
I2C_START_SEC_CODE I2C_STOP_SEC_CODE	■								
I2C_START_SEC_CODE_ISR I2C_STOP_SEC_CODE_ISR	■								
I2C_START_SEC_PBCFG_ROOT I2C_STOP_SEC_PBCFG_ROOT							■		
I2C_START_SEC_PBCFG I2C_STOP_SEC_PBCFG							■		
I2C_START_SEC_CONST_32BIT I2C_STOP_SEC_CONST_32BIT						■			
I2C_START_SEC_CONST_UNSPECIFIED I2C_STOP_SEC_CONST_UNSPECIFIED						■			
I2C_START_SEC_VAR_NOINIT_UNSPECIFIED I2C_STOP_SEC_VAR_NOINIT_UNSPECIFIED			■						
I2C_START_SEC_VAR_INIT_UNSPECIFIED I2C_STOP_SEC_VAR_INIT_UNSPECIFIED				■					
I2C_START_SEC_VAR_ZERO_INIT_UNSPECIFIED I2C_STOP_SEC_VAR_ZERO_INIT_UNSPECIFIED					■				
Application buffers passed to I2C									■
Notification called from I2C								■	

Table 4-3 Compiler abstraction and memory mapping

[1] I2C_VAR is used for pointer range.

4.3 Critical Sections

I2C has one critical Section:

> Interrupt locking blocks any interrupts and protects important global I2C variables from read / modify / write accesses.

4.4 Dependencies on SW modules

4.4.1 DET

The module I2C depends on the DET (by default) in order to report development errors. Detection and reporting of development errors can be enabled or disabled by the switch "Enable Development Error Detection" on the "General Settings" within the module I2C The DET can be replaced optionally by an equivalent component which is responsible to recognize development errors, if no DET component is available.

4.4.2 DEM

The module I2C depends on the DEM (by default) in order to report diagnostic events. Reporting of diagnostic events can be enabled or disabled by the switch " Dem Error Reporting" on the "General Settings" within the module I2C The DEM can be replaced optionally by an equivalent component which is responsible to recognize diagnostic events, if no DEM component is available.

4.4.3 MCU

The module MCU powers up the microcontroller’s peripherals at startup time. Since the peripherals are also containing the registers for I/O functionality they have to be activated if it is intended to use them.

4.4.4 PORT

The module Port enables the I2C lines at startup time. To operate the I2C properly, the Port driver has to be configured. The Port driver sets the Pins to the required values for the I2C to operate. The Pins used are SDA (serial data line) and SCL (serial clock). An open drain configuration is recommended.

4.4.5 SchM (Optional)

Beside the OSEK / AUTOSAR OS the Schedule Manager provides functions that module I2C calls at begin and end of critical sections.

4.4.6 Dependencies on HW modules

Please refer to the supplement documentation for details.

5 API Description

For an interfaces overview please see Figure 2-5.

5.1 Type Definitions

The external visible and usable types defined by the I2C are described in this chapter.

Type Name	C-Type	Description	Value Range
I2c_StatusType	enum	States of the I2C driver.	I2C_UNINIT, driver is not initialized.
			I2C_IDLE, driver is initialized and ready.
I2c_ResultType	uint16	The result / status of a channel.	I2C_BUSY, driver is processing a communication request.
			I2C_CHL_OK, channel is idle or has finished successfully.
			I2C_CHL_PENDING, channel is waiting to be serviced.
			I2C_CHL_WAITING, a follow up was used the channel is waiting for next transmission.
			I2C_CHL_CANCELED, channel transfer has been aborted by user.
			I2C_CHL_FAILED, channel communication failed.
			I2C_CHL_ERR_BUS, a bus error was detected by hardware.
			I2C_CHL_ERR_ARB, arbitration lost error.
			I2C_CHL_ERR_TX_DATA_NACK , Write: Data was sent but not acknowledged.
I2c_SeqResultType	enum	States of the sequence.	I2C_SEQ_OK, sequence finished normal. I2C_SEQ_PENDING, a sequence is in progress or queued. I2C_SEQ_NACK, sequence encountered a NACK signal. I2C_SEQ_FAILED, sequence failed due to error. I2C_SEQ_CANCELLED, sequence was cancelled by request.
I2c_DataType	uint8	Data format of the I2c used for transmission and reception.	Range(0-255)
I2c_DataLengthType	uint16	Type for any length	Range(1-65535)

Type Name	C-Type	Description	Value Range
		parameters.	
I2c_DataPtrType	uint8 *	Type for general buffer handling.	Pointer (near 16bit, far 32 bit).
I2c_DataConstPtrType	const uint8 *	Type for TX buffer handling.	Pointer (near 16bit, far 32 bit).
I2c_ChannelType	uint8	Type for channel identifier.	Contains the configuration of channels, buses and hardware units. Range(0-254)

Table 5-1 Type definitions

I2c_ConfigType

This is the configuration type of the I2c. A reference of this type is generated in the I2c_PBcfg.c file.

Struct Element Name	C-Type	Description	Value Range
I2c_SequenceCfg	I2c_SequenceConfigPtrType	Pointer reference to sequence configuration.	32bit.
I2c_SequenceCfg	I2c_ChannelConfigPtrType	Pointer to channel configuration.	32bit.
I2c_HwCfg	I2c_HardwareConfigPtrType	Pointer to hardware configuration.	32bit.
I2c_MaxSequence	uint8	Maximum number of configured sequences.	8bit.
I2c_MaxChannels	uint8	Maximum number of configured channels.	8bit.
I2c_MaxBus	uint8	Maximum number of configured bus units.	8bit.
I2c_MaxDlcs	uint8	Maximum number of configured hardware units.	8bit

Table 5-2 I2c_ConfigType

5.2 Interrupt Service Routines provided by I2C

Please refer to the supplement documentation for details.

5.3 Services provided by I2C

5.3.1 I2c_Init

Prototype	
`void` **`I2c_Init`** `( I2c_ConfigType* ConfigPtr )`	
Parameter	
`ConfigPtr`	Pointer to I2C configuration set.
Return code	
`void`	--
Functional Description	
The initialization must be called to operate the I2C driver. This applies to a power on reset or a normal reset. Other API services cannot be executed if the I2c_Init was not executed before and error checking is active.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is synchronous. > This function is non-reentrant. > This function returns without action if a check fails. > This function should not be called during a running operation.	
Expected Caller Context	
> Expected to be called in application context, during initialization phase.	

Table 5-3 I2c_Init

5.3.2 I2c_DeInit

Prototype	
`Std_ReturnType` **`I2c_DeInit`** `( void )`	
Parameter	
`void`	--
Return code	
`Std_ReturnType`	`E_OK`, success `E_NOT_OK`, request rejected.
Functional Description	
The de-initialization must be called to shut down the I2C driver. This applies to a system shutdown. It can also be used to reinitialize the system in case of a critical error.	

Particularities and Limitations
> Service ID: see table 'Service IDs'
> This function is synchronous.
> This function is non reentrant.
> This function returns without action if a check fails.
> This function should not be called during a running operation.
Expected Caller Context
> Expected to be called in application context, during shutdown phase.

Table 5-4 I2c_DeInit

5.3.3 I2c_SetupEB

Prototype

```
Std_ReturnType I2c_SetupEB ( I2c_ChannelType Channel,
                             I2c_DataConstPtrType TxDataBufferPtr,
                             I2c_DataPtrType RxDataBufferPtr,
                             I2c_DataLengthType Length)
```

Parameter

Channel	Channel used for transmission.
TxDataBufferPtr	Pointer to TX transmission data location.
RxDataBufferPtr	Pointer to RX transmission data location
Length	Length of the transmission.

Return code

Std_ReturnType	E_OK, success
	E_NOT_OK, request rejected.

Functional Description

The service conducts a buffer setup for later transmission of a channel. The slave is addressed by the configuration which belongs to the channel Id. The buffer pointer contains the location of the data which is written and received; the length parameter contains the number of data to be transmitted. Following buffer combinations are possible:

TxDataBufferPtr == NULL_PTR, RxDataBufferPtr = any valid address, will create a RX reception on this channel.

TxDataBufferPtr == any valid address, RxDataBufferPtr = NULL_PTR, will create a TX transmission on this channel.

TxDataBufferPtr == NULL_PTR, RxDataBufferPtr = NULL_PTR, will push default data on this channel.

TxDataBufferPtr == any valid address, RxDataBufferPtr = any valid address, is invalid and the service will reject the request.

Particularities and Limitations

> Service ID: see table 'Service IDs'.
> This function is synchronous.
> This function is non reentrant.
> This function returns without action if a check fails.

Expected Caller Context
> Expected to be called in application context, during operational phase.

Table 5-5 I2c_SetupEB

5.3.4 I2c_AsyncTransmit

Prototype	
`Std_ReturnType` **`I2c_AsyncTransmit`** `( I2c_SequenceType Sequence)`	
Parameter	
`Sequence`	Sequence used for data exchange.
Return code	
`Std_ReturnType`	`E_OK`, success `E_NOT_OK`, request rejected.
Functional Description	
The service conducts a data transmission according to the parameters provided to the `I2c_SetupEB()` service.	
Particularities and Limitations	
> Service ID: see table 'Service IDs' > This function is asynchronous. > This function is non reentrant. > This function returns without action if a check fails.	
Expected Caller Context	
> Expected to be called in application context, during operational phase.	

Table 5-6 I2c_AsyncTransmit

5.3.5 I2c_GetStatus

Prototype	
`I2c_StatusType` **`I2c_GetStatus`** `( void )`	
Parameter	
`void`	--
Return code	
`I2c_StatusType`	Please refer to chapter 5.1.
Functional Description	
Returns the current driver status.	

Particularities and Limitations
> Service ID: see table 'Service IDs'. > This function is synchronous. > This function is reentrant.
Expected Caller Context
> Expected to be called in application context, during operational phase.

Table 5-7 I2c_GetStatus

5.3.6 I2c_GetResult

Prototype	
`I2c_ResultType` **`I2c_GetResult`** `( I2c_ChannelType Channel )`	
Parameter	
`Channel`	Channel to query.
Return code	
`I2c_ResultType`	Please refer to chapter 5.1.
Functional Description	
Returns the current result of a channel operation. Channel result low byte indicates the operational status of the channel while the high byte offers additional information.	
Particularities and Limitations	
> Service ID: see table 'Service IDs'. > This function is synchronous. > This function is reentrant. > This function returns without action if a check fails.	
Expected Caller Context	
> Expected to be called in application context, during operational phase.	

Table 5-8 I2c_GetResult

5.3.7 I2c_GetSequenceResult

Prototype	
`I2c_ResultType` **`I2c_GetSequenceResult`** `( I2c_SequenceType Sequence )`	
Parameter	
`Sequence`	Sequence to query.
Return code	
`I2c_SeqResultType`	Please refer to chapter 5.1.
Functional Description	
Returns the current result of a sequence operation.	

Particularities and Limitations
> Service ID: see table 'Service IDs'. > This function is synchronous. > This function is reentrant. > This function returns without action if a check fails.
Expected Caller Context
> Expected to be called in application context, during operational phase.

Table 5-9 I2c_GetSequenceResult

5.3.8 I2c_Cancel

Prototype	
`void` **`I2c_Cancel`** `( I2c_SequenceType Sequence )`	
Parameter	
`I2c_SequenceType`	Sequence to cancel.
Return code	
`void`	--
Functional Description	
The service aborts an ongoing transmission/reception of data. This service is executed only on pending or active sequences.	
Particularities and Limitations	
> Service ID: see table 'Service IDs'. > This function is synchronous. > This function is non reentrant. > This function returns without action if a check fails.	
Expected Caller Context	
> Expected to be called in application context, during operational phase.	

Table 5-9 I2c_Cancel

5.3.9 I2c_GetVersionInfo

Prototype	
`void` **`I2c_GetVersionInfo`** `(Std_VersionInfoType VersioninfoPtr )`	
Parameter	
`Std_VersionInfoType`	--
Return code	
`void`	--

Functional Description
Returns the version information of the module.
Particularities and Limitations
> Service ID: see table 'Service IDs'. > This function is synchronous. > This function is reentrant. > This function returns without action if a check fails.
Expected Caller Context
> Expected to be called in application context, during operational phase.

Table 5-10 I2c_GetVersionInfo

5.3.10 I2c_SetHandling

Prototype	
`void` **`I2c_SetHandling`**`(I2c_HandlingType Handling)`	
Parameter	
`Std_VersionInfoType`	Please refer to chapter 5.1.
Return code	
`void`	--
Functional Description	
Sets the handling of the main function service: I2C_BURST, the timeout detection of the main function service is disabled. I2C_RECURRENT, the timeout detection of the main function service is enabled.	
Particularities and Limitations	
> Service ID: see table 'Service IDs'. > This function is synchronous. > This function is reentrant.	
Expected Caller Context	
> Expected to be called in application context, during operational phase.	

5.3.11 I2c_MainFunction

Prototype	
`void` **`I2c_MainFunction`** `( void )`	
Parameter	
`void`	--
Return code	
`void`	--

Functional Description
The Main Function service handles cyclic procedures. It is used to update the driver status depending on the amount of pending channels. This function is required after each transmission service call for schedule update. Keep in mind that the sequence queue is not processed if this service is not called.
Particularities and Limitations
> Service ID: see table 'Service IDs'. > This function is synchronous. > This function is non reentrant. > This function returns without action if a check fails.
Expected Caller Context
> Expected to be called in application context, during operational phase.

Table 5-11 I2c_MainFunction

5.4 Services used by I2C

In the following table services provided by other components, which are used by the I2C are listed. For details about prototype and functionality refer to the documentation of the providing component.

Component	API
DET	Det_ReportError
DEM	Dem_SetEventStatus

Table 5-12 Services used by the I2C

5.5 Callback Functions

The I2C does not have any callbacks which can be invoked by other modules.

5.6 Configurable Interfaces

5.6.1 Notifications

At its configurable interfaces the I2C defines notifications that can be mapped to callback functions provided by other modules. The mapping is not statically defined by the I2C but can be performed at configuration time. The function prototypes that can be used for the configuration have to match the appropriate function prototype signatures, which are described in the following sub-chapters.

5.6.1.1 I2c_SequenceEndNotification

Prototype	
`void (* I2c_SequenceEndNotification) ( void )`	
Parameter	
`void`	--

Return code	
`void`	--
Functional Description	
Sequence transfer end notification function. Can be configured in the configuration tool.	
Particularities and Limitations	
> It is advised to keep the code execution in the notification function as short as possible.	
Call context	
> Interrupt context.	

Table 5-13 I2c_SequenceEndNotification

5.6.1.2 I2c_SequenceErrorNotification

Prototype	
`void (* I2c_SequenceErrorNotification) ( void )`	
Parameter	
`void`	--
Return code	
`void`	--
Functional Description	
Sequence transfer error notification function. If the I2c detects an error, this function notifies the application about a communication failure. Can be configured in the configuration tool.	
Particularities and Limitations	
> It is advised to keep the code execution in the notification function as short as possible.	
Call context	
> Interrupt context.	

Table 5-14 I2c_SequenceEndNotification

6 Configuration

In the I2C the attributes can be configured according to/ with the following methods/ tools:

> Configuration in DaVinci Configurator (Base/Pro).

6.1 Configuration Variants

The I2C supports the configuration variants

> VARIANT-PRE-COMPILE

The configuration classes of the I2C parameters depend on the supported configuration variants. For their definitions please see the I2C_bswmd.arxml file.

6.2 Configuration with DaVinci Configurator Pro

Within the configuration tool the module I2C can be located in the “Basic Editor”, see Figure:

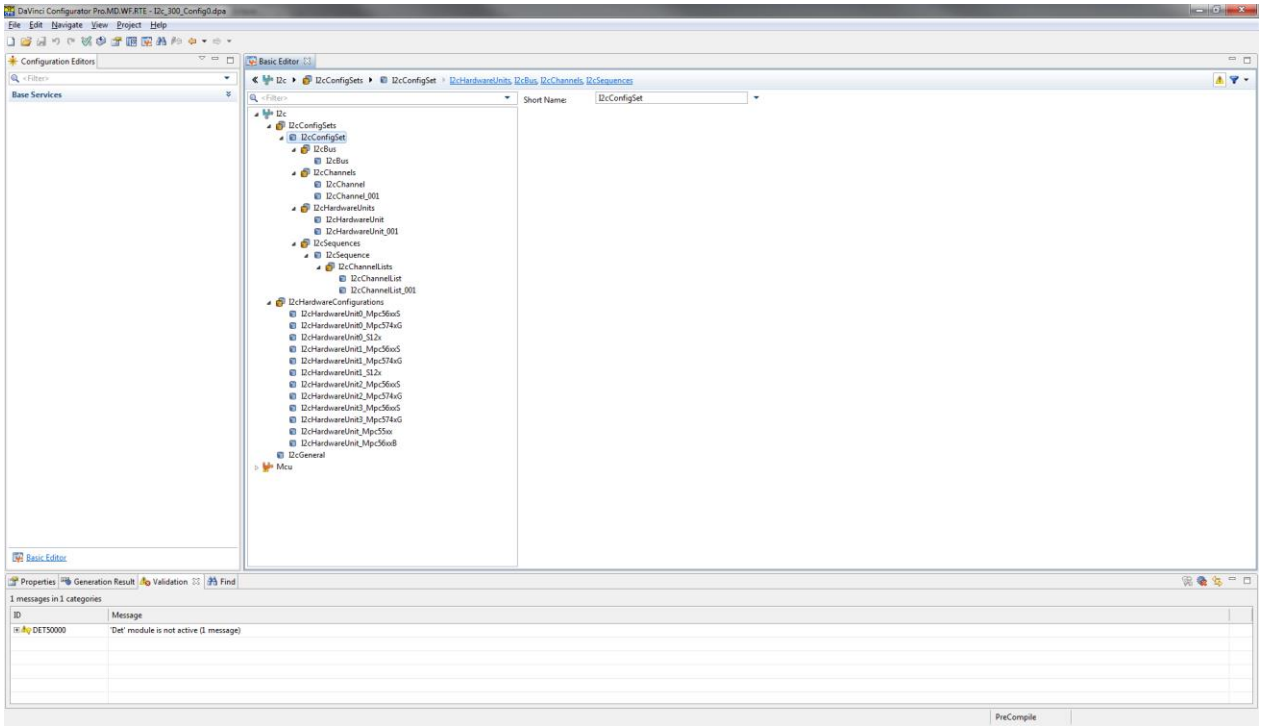


Figure 6-1 Configurator Basic Editor View.

6.2.1 Start configuration of I2C

The parameters are described in the BSWMD module description and can be viewed by selecting the parameter and the “Properties” Window.

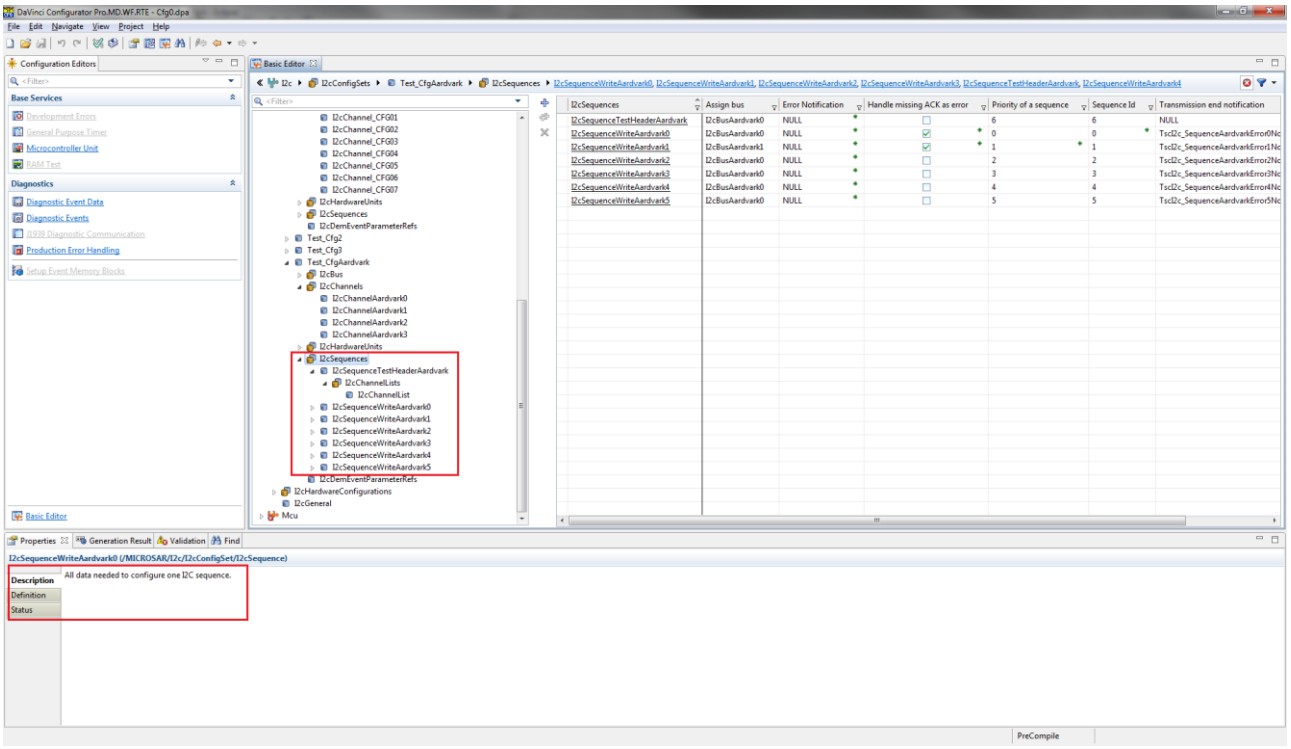


Figure 6-2 Parameter description

Please refer to the supplement documentation for details.

6.2.1.1 Configuration sets

The driver supports multiple configuration sets. To use another set, the driver must de-initialize and re-initialize with the new configuration set pointer. The configuration set may have different settings. All parameters below the I2cConfigSet container are affected.

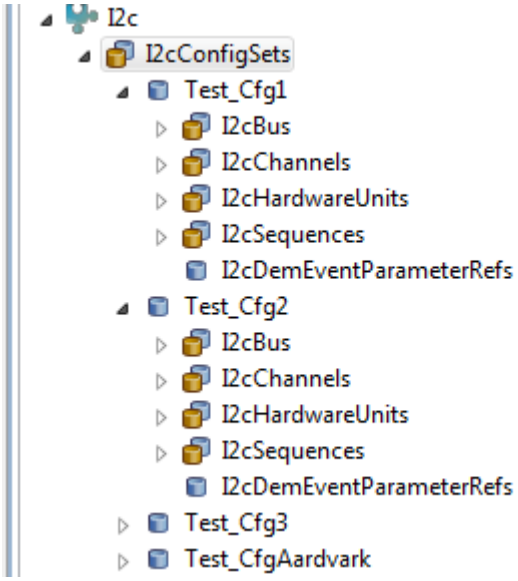


Figure 6-3 Configure multiple configuration sets

6.2.1.2 Channels

Channels are the smallest data unit on the I2C bus. The channels are created to serve different purposes.

Example

An application can prepare a buffer which is filled with data for an external device. Most external devices need a register or an address to be set prior to the data payload. It is possible to set up a single data transfer by adjusting the buffer pointers. In fact this is not an easy task and to make an abstraction of this the channels can be used.

In this example the address and the data are abstracted by two channels. One channel is setting up the address or register, the data channel then programs the data into this selected register. For this the API I2c_SetupEB has to be called for each channel located in a sequence, here two times (see 6.2.2).

Also a channel can carry additional properties like chaining (see 6.2.2.1) which handles a restart condition.

6.2.1.3 Channel lists

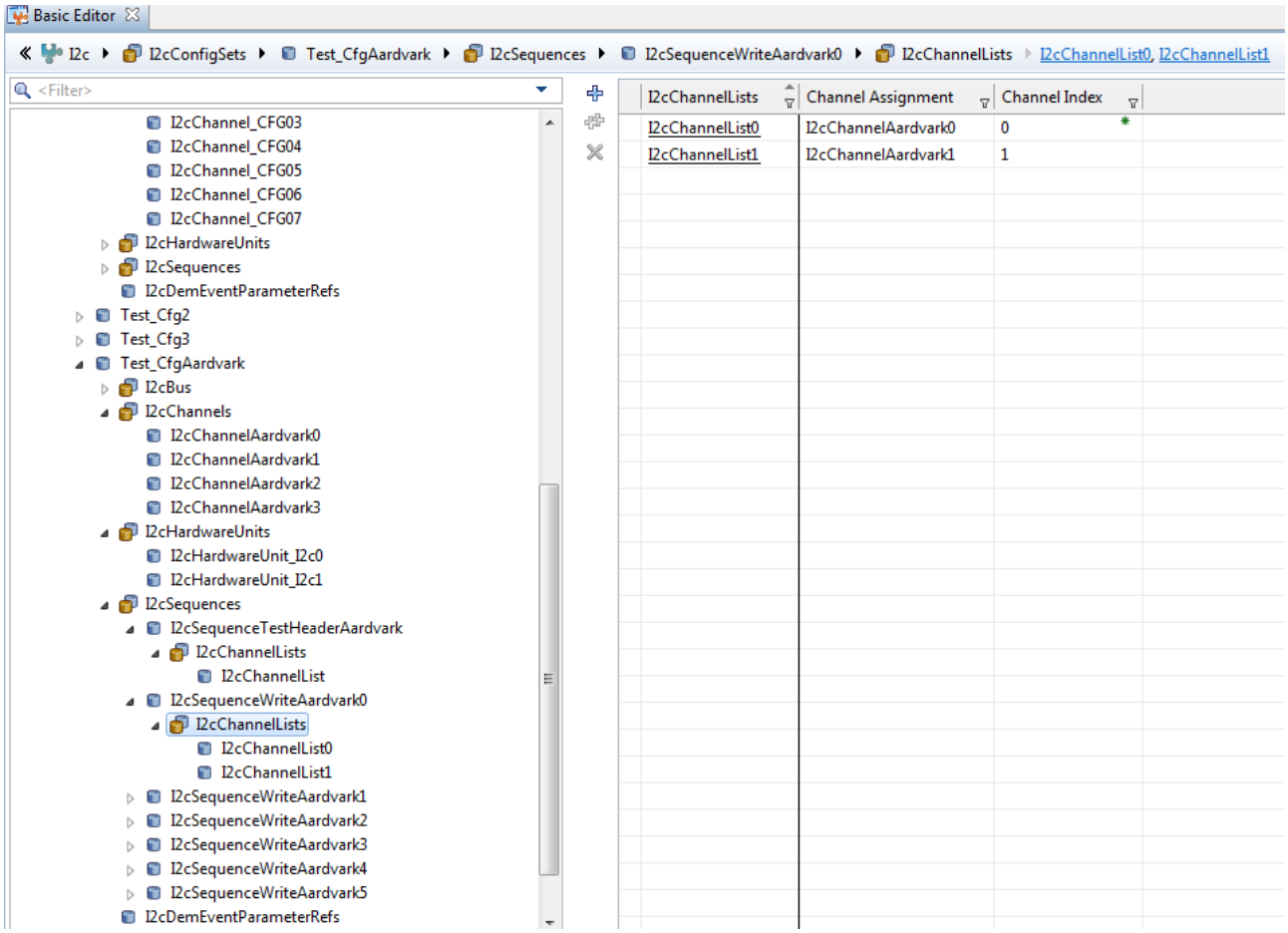


Figure 6-4 Channel lists



Basic Knowledge
A channel list consists of one or more channels which are transferred by a sequence according to their index (index 0 is always the first channel).

6.2.1.4 Sequences



Basic Knowledge
Sequences are the smallest atomic unit which is transferred on the I2C bus. Although assembled by channels, sequences are not interruptible. The functionality of a sequence and the contained channels is explained on the next page.

6.2.2 Communication example



Example

For this example an external EEPROM unit is chosen (MICROCHIP 24AA128). For this unit three sequences are required:

- Eep_WriteSequence with channels
  - I2cChannelWriteCom
  - I2cChannelWriteData (is chained)
- Eep_ReadSequence and
  - I2cChannelReadCom
  - I2cChannelReadData
- Eep_AckSequence (handle NACK as error" must be off).
  - I2cChannelAck

All other channels are not chained.

After setting up the channels:

- I2cChannelWriteCom gets the internal EEP address where the data shall be stored ("PAGE WRITE"). The chain channel feature enables the next channel to be written without a "restart condition".
- I2cChannelWriteData gets the data which shall be stored ongoing from this address.
- I2cChannelReadCom gets the internal EEP address where the data is stored with write access ("RANDOM READ").
- I2cChannelReadData is set up with a buffer which receives incoming data.

The sequence can now be transmitted:

`I2c_AsyncTransmit(Eep_WriteSequence)`

To ensure the data has been written, the Eep_AckSequence must be applied and transmitted as long the sequence status is `I2C_SEQ_NACK`. The I2c_MainFunction must be triggered during this procedure. The number of `NACK` events depends on the processor speed and thus how often the chip is queried.

The read sequence can now be executed and the written data verified:

`I2c_AsyncTransmit(Eep_ReadSequence)`

See next page for transaction logs.

6.2.2.1 Logged transaction according to the given example

SP	50	Write Transaction	01 00 05 01 50 00 0C 0A 0F 0E A5 5A

Figure 6-5 PAGE WRITE transaction.

SP	50*	Write Transaction	AA*
SP	50*	Write Transaction	AA*
SP	50*	Write Transaction	AA*
SP	50*	Write Transaction	AA*
SP	50	Write Transaction	AA

Figure 6-6 Data polling

S	50	Write Transaction	01 00
SP	50	Read Transaction	05 01 50 00 0C 0A 0F 0E A5 5A

Figure 6-7 RANDOM READ transaction

Expert Knowledge
A normal channel will trigger always a restart condition (S[TART]...S...S...[STO]P) but a chained channel does not (S...x...x...x...P, x marks the beginning of a new channel which is not visible in the trace).

6.2.2.2 Possibilities of the driver (transfer examples).

Expert Knowledge
The first channel of a sequence ignores the property “chain channel”. This is not allowed according to the I2C specification. So if a channel is chained, it affects every channel but not the first!

Another exclusion of the chained property is a direction change. If this first channel starts as master transmitter and changes to master receiver. This is necessary due to the header by which contains the direction. A restart condition is triggered instead.

For the first three examples the 7-bit representation of the address byte is used.

Example 1:



Example
Consider a sequence with one channel which addresses a device at 0x20. Ten bytes shall be transmitted. Channel is not chained.



Practical Procedure
Prepare a data buffer and call the API's:

- I2c_SetupEB(Channel, Buffer, NULL_PTR, 10)
- I2c_AsyncTransmit(Sequence)



Note
Output to the bus:

```
[S]tart 0x20 Byte0.. Byte9 Sto[P] (Channel)
```

Example 2:



Example
Consider a sequence with two channels which address a device at 0x20. Two bytes and eight bytes shall be transmitted. Channels are not chained.



Practical Procedure
Prepare a data buffer and call the API's:

- I2c_SetupEB(Channel10, Buffer0, NULL_PTR, 2)
- I2c_SetupEB(Channel11, Buffer1, NULL_PTR, 8)
- I2c_AsyncTransmit(Sequence)



Note
Output to the bus:

```
[S]tart   0x20 Byte00.. Byte01         (Channel0)
Re[S]tart 0x20 Byte10.. Byte17 Sto[P]  (Channel1)
```

Example 3:



Example
Consider a sequence with three channels which address a device at 0x20. Two bytes , eight and four bytes shall be transmitted. Channel two is the only chained one.



Practical Procedure
Prepare a data buffer and call the API's:

- I2c_SetupEB(Channel10, Buffer0, NULL_PTR, 2)
- I2c_SetupEB(Channel11, Buffer1, NULL_PTR, 8)
- I2c_SetupEB(Channel12, Buffer2, NULL_PTR, 4)
- I2c_AsyncTransmit(Sequence)



Note
Output to the bus:

```
[S]tart   0x20 Byte00.. Byte01         (Channel0)
Re[S]tart 0x20 Byte10.. Byte17.. Byte20.. Byte 23 Sto[P]
          < A ><  Channel11  >   <  Channel12   >
```

As the examples show, every channel begins with the output of a start condition `[S]` and the address `[A] = 0x20` of a channel. If the channel has a different address, a different device is addressed. Switching to a channel without the property "chained" leads to a restart condition `[S]` and an address transmission. Now the switch to a channel with "chained" property continues transmission on the bus without triggering a restart condition and transmitting a header. A sequence is always concluded with a stop condition `[P]`.

Example 4:
Now the 8-bit representation of the address byte is used.



Example
Consider a sequence with two channels which address a device at 0x20. Two bytes and eight bytes shall be transmitted. Channel11 is chained. During the change over from Channel10 to Channel11, the direction is changed.



Practical Procedure
Prepare a data buffer and call the API's:

- I2c_SetupEB(Channel10, Buffer0, NULL_PTR, 2) (master transmitter)
- I2c_SetupEB(Channel11, NULL_PTR, Buffer1, 8) (master receiver)
- I2c_AsyncTransmit(Sequence)



Note
Output to the bus:

```
[S]tart   0x40 Byte00.. Byte01         (Channel0)
Re[S]tart 0x41 Byte10.. Byte17 Sto[P]  (Channel1)
```

Example 5:



Example

Consider a sequence with two channels which address a device at 0x20. Two bytes and eight bytes shall be transmitted. Channel11 is chained. During the change over from Channel10 to Channel11, the direction is kept.



Practical Procedure

Prepare a data buffer and call the API's:

- I2c_SetupEB(Channel10, Buffer0, NULL_PTR, 2) (master transmitter)
- I2c_SetupEB(Channel11, Buffer1, NULL_PTR, 8) (master transmitter)
- I2c_AsyncTransmit(Sequence)



Note

Output to the bus:

```
[S]tart   0x40 Byte00..Byte01 .. Byte10.. Byte17  Sto[P]
          < A ><  Channel10 >    <  Channel11  >
```

Example 6:



Example
Consider a sequence with two channels which address a device at 0x20. Two bytes and eight bytes shall be transmitted. Channel11 is chained and shall have the address 0x22. During the change over from Channel10 to Channel11, the direction is kept.



Practical Procedure
Prepare a data buffer and call the API's:

- I2c_SetupEB(Channel10, Buffer0, NULL_PTR, 2) (master transmitter)
- I2c_SetupEB(Channel11, Buffer1, NULL_PTR, 8) (master transmitter)
- I2c_AsyncTransmit(Sequence)



Note
Output to the bus:

```
[S]tart   0x40 Byte00..Byte01 .. Byte10.. Byte17  Sto[P]
          < A ><  Channel10 >    <  Channel11  >
```

Due to address bypass, the channel address is ignored! Referring to **Example 4**, the correct address will be transmitted due to a restart condition.

7 Glossary and Abbreviations

7.1 Deviations

The I2C driver is not specified originally by AUTOSAR. It is a simple AUTOSAR conform specification.

7.2 Additions/ Extensions

7.2.1 I2C sequence queue feature

Sequences can be queued up to the “Queue Depth” even if a sequence blocks the bus. The processing of this queue is handled in the `I2c_MainFunction`. If the queue runs full, the I2c_AsyncTransmit service is declined. At the moment there is no other possibility to check if the queue runs full.

7.2.2 I2C chain channel feature

A chained channel is only used for data transmission. It initially sets a start condition and address but next time the channel is used, it continues sending data. A chained channel must not be the first channel transmitted by a sequence.

7.2.3 I2C extended addressing

Use this feature if your device is mapped beyond address of 127. The driver takes care about the addressing according to the I2C specification.

7.3 Glossary

Term	Description
DaVinci Configurator Pro	Configuration and generation tool for MICROSAR components
PORT	A Port represents a whole configurable port on this microcontroller (typically controlled by one hardware register set).
I2C	Inter Integrated Circuit, also known as IIC or I^2C
MCU	Micro controller unit driver. Configures the processor (clocks, PLL)
EAD	Embedded Architecture Designer; generation tool for MICROSAR components

Table 7-1 Glossary

7.4 Abbreviations

Abbreviation	Description
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSW	Basis Software
DEM	Diagnostic Event Manager
DET	Development Error Tracer

EAD	Embedded Architecture Designer
ECU	Electronic Control Unit
HIS	Hersteller Initiative Software
ISR	Interrupt Service Routine
MICROSAR	Microcontroller Open System Architecture (the Vector AUTOSAR solution)
PPORT	Provide Port
RPORT	Require Port
RTE	Runtime Environment
SRS	Software Requirement Specification
SWC	Software Component
SWS	Software Specification

Table 7-2 Abbreviations

8 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com